CS6888 Software Analysis and its Applications HW2

Sravanth Chowdary Potluri
und5uv@virginia.edu
University Of Virginia

I. Benchmark Design Considerations

This section presents the design considerations for the 10-program benchmark suite. Our objective was to create a diverse, representative, and concise set of programs that highlight both the strengths and limitations of symbolic execution using KLEE. Each program was developed to satisfy the following criteria:

1) **Original:** The programs are independently developed for this assignment and are not directly taken from existing KLEE tutorials.
2) **Distinct:** Each program targets a unique language construct or behavior (e.g., arithmetic properties, loop iterations, recursion, pointer arithmetic, looping, branching etc).
3) **Concise:** All programs are under 50 lines, ensuring a clear focus on one specific feature.
4) **Targeted:** The program filename reflects its intended feature (e.g., `1-divisibility-check-sym.c` for testing arithmetic properties). The number prefix indicates the subset of programs under which the program falls in i.e 1 for the first subset and 2 for the second subset.
5) **Diverse:** The suite covers a range of different and unique programming constructs that are common in real-world applications.
6) **Representative:** Each example models a realistic scenario, whether it is a typical validation routine or a worst-case scenario that stresses the symbolic engine. Even though the last five programs are designed to create worst-case scenarios—forcing KLEE to solve a large number of constraints or to explore an enormous number of paths—they remain representative of real-world applications. For instance, many systems employ complex pattern matching, state machines, or recursive algorithms for tasks such as protocol validation, input parsing, or configuration analysis. In these contexts, the underlying logic often naturally gives rise to extensive branching and intricate constraint systems, mirroring the challenges presented in our benchmark.

We divided the benchmark into two subsets. The first five programs are those for which KLEE efficiently achieves full branch coverage, while the latter five are designed to expose KLEE's limitations (e.g., due to exponential branching, recursion explosion, or memory safety issues).

A. Program 1: `1-divisibility-check-sym.c`

This program uses a single symbolic integer to check if it is divisible by 3. It is original and focused on a simple arithmetic property. Its design is concise and targeted, and it is representative of common numerical validations. Since the constraints are straightforward, KLEE is able to achieve full branch coverage, illustrating one of its strengths.

B. Program 2: `1-max-check-sym.c`

In `max-check-sym.c`, two symbolic integers are compared to determine the relational ordering (greater than, less than, or equal). This program is distinct from Program 1 as it involves comparing two values rather than evaluating a single arithmetic property. Its simplicity and direct constraint formulation allow KLEE to fully explore all three possible outcomes, making it both representative and efficient.

C. Program 3: `1-loop-iteration-sym.c`

This program uses a symbolic loop bound to control iteration, where the sum is computed by conditionally adding or subtracting the loop index. It is original in demonstrating symbolic control flow through loops and is concise and targeted. The program is representative of typical iterative constructs in software, and its moderate constraints enable full path exploration by KLEE.

D. Program 4: `1-struct-check-sym.c`

Here, a custom `Person` structure with two fields (age and score) is used, with symbolic constraints applied to each field. This program is distinct because it operates on a composite data type rather than on primitive values. Its design is concise, targeted, and representative of real-world validation scenarios, with KLEE efficiently covering all feasible paths.

E. Program 5: `1-sorted-check-sym.c`

This program checks whether a symbolic array of five integers is sorted in non-decreasing order. It is original and distinct as it considers ordering constraints across multiple elements, and it is concise and focused. The sortedness property is representative of common data integrity checks. KLEE is able to fully cover the branches in this controlled environment, demonstrating its strength in handling simple array properties.

F. Program 6: `2-huge-branching-sym.c`

Designed to push KLEE into a state of exponential path explosion, this program uses a 24-element symbolic character array. Each element contributes a binary branch, resulting in up to 2^{24} potential paths. The design is original and distinct in its focus on exponential branching. Although the program is concise and targeted, the vast state space makes it a clear example where KLEE struggles, highlighting limitations in handling extreme branching scenarios.

G. Program 7: `2-recursion-explosion-sym.c`

This program implements a naive recursive Fibonacci function with a symbolic input constrained to values up to 25. The recursion leads to exponential growth in the call tree. It is original, distinct (focused on recursion rather than iterative loops), and concise. The targeted nature of the recursion explosion is representative of worst-case scenarios in symbolic execution. KLEE's performance deteriorates as it must contend with an exponentially growing number of recursive calls, showcasing its limitations in such cases.

H. Program 8: `out-of-bounds-sym.c`

In this program, a deliberate out-of-bounds memory access is introduced by looping 100 times over an array of 10 elements. The error occurs because the loop index exceeds the allocated size of the array, leading to attempts to access memory that was not allocated to the array. KLEE's runtime detects this violation of memory safety and terminates the corresponding execution paths. By inspecting one of the generated test files with `ktest-tool`, we can see that only the first 10 bytes of `arr` are concretely assigned, and any index beyond 9 triggers an immediate memory error. Although this program is concise and targeted, its design purposefully triggers undefined behavior, demonstrating a practical limitation of symbolic execution: incomplete coverage due to early path termination caused by memory errors. This scenario is representative of real-world bugs, such as buffer overflows, where improper bounds checking can lead to security vulnerabilities or system crashes.

I. Program 9: `2-multi-pointer-arithmetic-sym.c`

This version attempts to stress KLEE by introducing multiple iterations where a symbolic index is used to access an array element. In the revised design, a loop with 10 iterations creates a significantly larger state space by introducing 10 independent symbolic variables. Despite the original two-variable version completing quickly, this multi-iteration approach is intended to result in a combinatorial explosion. Although the design is concise and targeted, the increased symbolic complexity makes it a candidate for KLEE's struggle—if not in complete coverage, then at least in terms of increased solver time.

J. Program 10: `2-multi-var-sym.c`

The final program introduces an array of 10 symbolic characters, where each character is constrained to be one of four possible values. The goal is to check a combinatorial condition: exactly 5 elements must equal a specific value (e.g., 'A'). This design is original and distinct from earlier examples because it combines disjunctive constraints with a counting condition, leading to a worst-case of 4^{10} possible combinations. It is concise, targeted, and highly representative of combinatorial problems in real systems. Due to the exponential growth in possibilities, KLEE is likely to struggle with full coverage within the allotted time.

TABLE I
KLEE COVERAGE REPORT FOR BENCHMARK PROGRAMS

Program File	# Tests	Coverage (%)
divisibility-check-sym.c	2	100
max-check-sym.c	3	100
loop-iteration-sym.c	10	100
struct-check-sym.c	3	100
sorted-check-sym.c	5	100
huge-branching-sym.c	∞	NA
recursion-explosion-sym.c	∞	NA
out-of-bounds-sym.c	1	50
multi-pointer-arithmetic-sym.c	∞	NA
multi-var-sym.c	∞	NA

II. COVERAGE REPORT EXPLANATION

The table above clearly distinguishes two subsets of benchmark programs. The first subset—comprising `divisibility-check-sym.c`, `max-check-sym.c`, `loop-iteration-sym.c`, `struct-check-sym.c`, and `sorted-check-sym.c`—achieves 100% branch coverage with a limited number of test cases (ranging from 2 to 10). This is because these programs feature relatively simple constraints, bounded loops, or straightforward comparisons that allow KLEE to efficiently explore all feasible paths.

In contrast, the second subset is intentionally designed to stress the symbolic engine, causing a combinatorial explosion, deep recursion, or memory safety issues. Programs such as `huge-branching-sym.c`, `recursion-explosion-sym.c`, `multi-pointer-arithmetic-sym.c`, and `multi-var-sym.c` result in KLEE generating an effectively infinite number of test cases (denoted as ∞) and report "NA" for branch coverage. This is because, although KLEE may approach full branch coverage by exploring almost all paths, it never fully terminates due to the exponential growth in the state space, leading to incomplete exploration within the allotted time. The only exception in this subset is `out-of-bounds-sym.c`, which, due to early termination from memory errors, generates very few tests and achieves only partial coverage (50%).

Due to the resource-intensive nature of the second subset, these programs are commented out in `run_all.sh` and should only be run when necessary, as they may overwhelm the system or cause KLEE to run indefinitely.

III. ANALYSIS

A. Key Observations

Our benchmark revealed that KLEE excels at handling programs with relatively simple symbolic con-

straints—such as arithmetic comparisons and bounded loops—where the state space remains small and the constraints are linear or easily solvable. In contrast, programs that introduce combinatorial explosion (e.g., `huge-branching-sym.c` and `multi-var-sym.c`) or deep recursion (e.g., `recursion-explosion-sym.c`) lead to an exponential increase in the number of paths. This vast state space, combined with complex constraint systems, severely challenges KLEE's underlying solvers. In particular, when multiple independent symbolic conditions are combined—such as in state machines or multi-variable arrays—the number of potential states becomes enormous, causing KLEE to either time out or report incomplete branch coverage. Additionally, memory safety issues, as seen in `out-of-bounds-sym.c`, cause early path termination, further reducing coverage.

B. Surprises in KLEE's Behavior

One of the surprising aspects was KLEE's efficiency on programs that one might expect to be challenging. For instance, even with non-trivial recursive functions or multiple arithmetic comparisons, KLEE managed to achieve full branch coverage in some cases. However, it was also notable how rapidly performance degraded when the number of symbolic choices increased even modestly. The sheer number of paths in the state-machine and combinatorial examples was far greater than anticipated, underscoring the sensitivity of symbolic execution to the underlying combinatorial explosion. Overall, while KLEE handled simple constraints with ease, the exponential growth in the state space for certain constructs was a clear indication of its limitations.

C. Conjectures on Handling Poorly Solved Constructs

For constructs that KLEE struggles with, such as exponential branch explosion and deep recursion, the primary issue is the combinatorial growth of the state space. In these cases, every additional symbolic decision multiplies the number of potential execution paths, overwhelming the solver. Furthermore, when constraints become highly interdependent—as in state machines where the global state evolves with each symbolic choice—the complexity of the constraint system increases dramatically, making it difficult for the SMT solvers to simplify and resolve. Additionally, memory safety issues trigger immediate path termination, preventing KLEE from exploring potentially interesting behaviors beyond the violation. In summary, the intrinsic limitations of constraint solvers and the exponential nature of the state space are the key reasons behind KLEE's struggles.

D. Prospects for Concolic Execution

Below, we discuss how concolic execution might fare for each of the five programs in the second subset:

Program 6: `huge-branching-sym.c`
In this program, a 24-element symbolic character array is used to generate up to 2^{24} potential paths. The exponential branch explosion makes it impractical for KLEE to exhaustively explore all paths. Concolic execution could help by executing one concrete instance at a time and only perturbing inputs to drive new behaviors, thus avoiding the combinatorial explosion. However, even with concolic execution, the sheer number of independent symbolic choices might still overwhelm the approach if not properly guided.

Program 7: `recursion-explosion-sym.c`
This program implements a naive recursive Fibonacci function with an increased upper bound (e.g., $n \leq 25$), leading to exponential recursion. Here, the recursive call tree grows very quickly, and KLEE struggles with the vast number of recursive calls. Concolic execution can prioritize concrete paths that avoid the worst-case recursion depth, thereby reducing the effective state space. However, because the recursion inherently causes exponential growth, concolic methods may only partially alleviate the issue.

Program 8: `out-of-bounds-sym.c`
In `out-of-bounds-sym.c`, a deliberate memory safety violation occurs by looping far beyond the array's allocated size. While this program primarily highlights KLEE's memory safety checks, concolic execution might help by executing concrete inputs that do not trigger the out-of-bound access. Nevertheless, if the chosen concrete path still encounters the invalid memory access, the benefits of concolic tracking are limited because the error is inherent to the program's design.

Program 9: `multi-pointer-arithmetic-sym.c`
This revised program uses a loop with 10 independent symbolic indices, leading to an enormous state space (on the order of 256^{10} combinations in the worst case). Concolic execution could potentially reduce this complexity by focusing on one concrete combination at a time and iteratively exploring alternatives. The effectiveness would depend on the ability of the concolic engine to prioritize high-impact perturbations, though the inherent combinatorial explosion may still render complete coverage infeasible.

Program 10: `multi-var-sym.c`
Here, an array of 10 symbolic characters is constrained to four possible values each, with a combinatorial condition requiring exactly five occurrences of a specific value. This results in roughly 4^{10} possible assignments. Concolic execution can be beneficial by guiding the exploration toward paths that satisfy the counting constraint, rather than blindly exploring all combinations. Nonetheless, the combinatorial nature of the constraint remains a challenge that may limit the overall effectiveness of the approach.

In summary, concolic execution holds significant promise in reducing the overhead of path exploration by incrementally generating concrete test cases and focusing on high-probability paths. However, for problems characterized by exponential constraint growth or fundamental memory safety issues, even concolic execution may struggle to fully overcome these limitations.

This homework and report are completed with the help of resources provided along with the homework. This report also makes use of o3-mini to refine the content.